Advanced Data Structure 2026

U2000

National Taipei University

Midterm

1. Select **all correct** statements about **sorting**: (10%)
   (a) **Insertion sort** encounters its best case if the input list is in **increasing order**.
   (b) If average-case time complexity is the main consideration, **quick sort** is a good choice.
   (c) If worst-case time complexity is important, both **heap sort** and **merge sort** are more suitable than **quick sort**.
   (d) **Quick sort** is not a **stable** sorting algorithm.
   (e) The time complexity lower bound for any comparison-based sorting algorithm is $\Omega(n \log n)$.

   (a),(b),(c),(d),(e) or (b),(c),(d),(e)

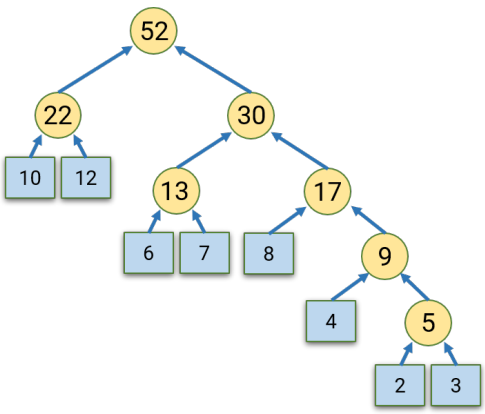
2. Consider an array of 8 elements being sorted using **quicksort**. It has just finished the first pass of partitioning and pivot swapping, thus the original array is now [4, 5, 8, 3, 10, 18, 12, 20]. Write down **all possible elements** that could have been the pivot in the first pass (8%)
   10 and 20

3. Use **Straight Radix Sort** to sort the following 8 numbers in ascending order:

$$(29, 662, 297, 114, 436, 780, 255, 8).$$

   After each bucket-sorting pass (for each digit), write down the intermediate sequence obtained after that pass. (8%)
   Pass 1: $(780, 662, 114, 255, 436, 297, 8, 29)$
   Pass 2: $(8, 114, 29, 436, 255, 662, 780, 297)$
   Pass 3: $(8, 29, 114, 255, 297, 436, 662, 780)$

4. Assume you have split your data into 8 segments for external sorting, with the sizes of the 8 segments being (10, 4, 3, 6, 12, 7, 2, 8). Draw the final optimal merge tree to merge the segments. Place the shorter segment on the left when building the tree. (8%)



5. You are given a static hash table with 17 buckets (indexed from 0 to 16). Each bucket has only one slot. Insert the following 10 keys into the table in the given order:

$$(20, 37, 54, 3, 71, 10, 5, 27, 22, 44).$$

For each question, show the final hash table after all keys have been inserted.

(a) Use **linear probing** to resolve collisions with the hash function: (8%)

$$(h(k) + j) \bmod 17, \qquad j = 0, 1, 2, 3, \ldots.$$

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
			20	37	54	3	71	5	22	10	27	44				

(b) Use **quadratic probing** to resolve collisions with the hash function: (8%)

$$h(k) \bmod 17, \quad (h(k) + i^2) \bmod 17, \quad (h(k) - i^2) \bmod 17 \qquad i = 0, 1, 2, 3 \ldots.$$

That is, the hash functions are $h(k) \bmod 17$, $(h(k)+1^2) \bmod 17$, $(h(k)-1^2) \bmod 17$, $(h(k)+2^2) \bmod 17$, $(h(k)-2^2) \bmod 17$, ...

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
		54	20	37	5	22	3		44	10	27					71

6. Assume you are using **dynamic hashing** to store data. Each bucket has **two** slots. The hash function converts a key into its **4-bit binary representation** and uses the Least Significant Bits (LSB) for indexing, as shown in Figure 1(a).

(a) When using **directory-based dynamic hashing**, given the current hash table shown in Figure 1(b), show how the table changes when a new key, 3, is inserted. (8%)

(b) When using **directory-less dynamic hashing**, given the current hash table shown in Figure 1(c), show how the table changes when a new key, 3, is inserted. (8%)

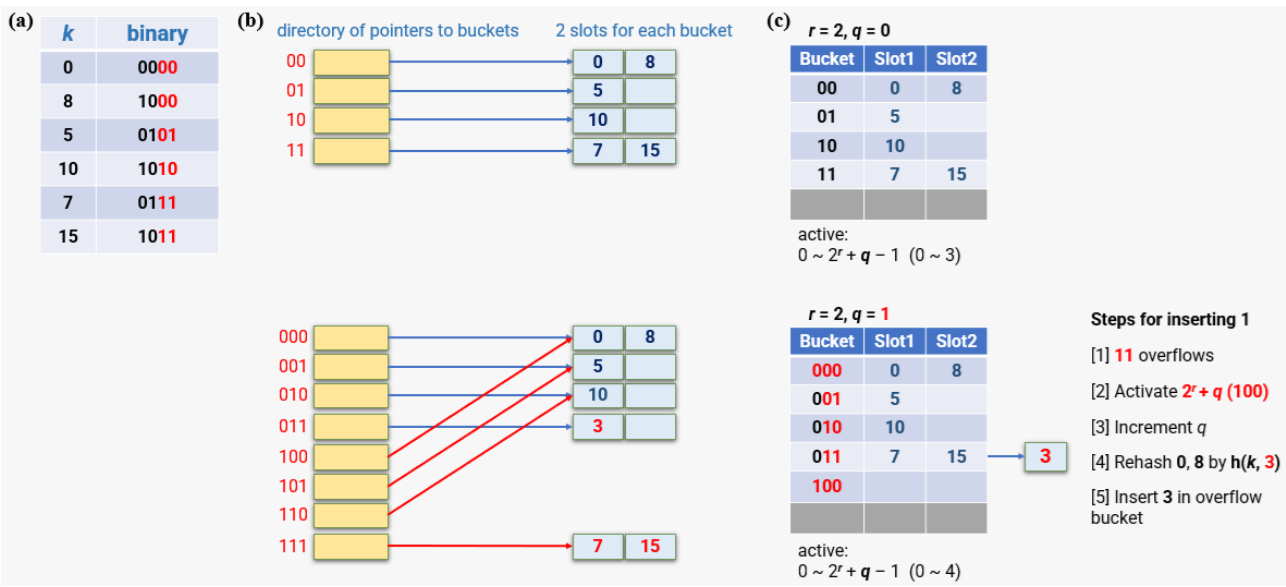


Figure 1: Dynamic hashing

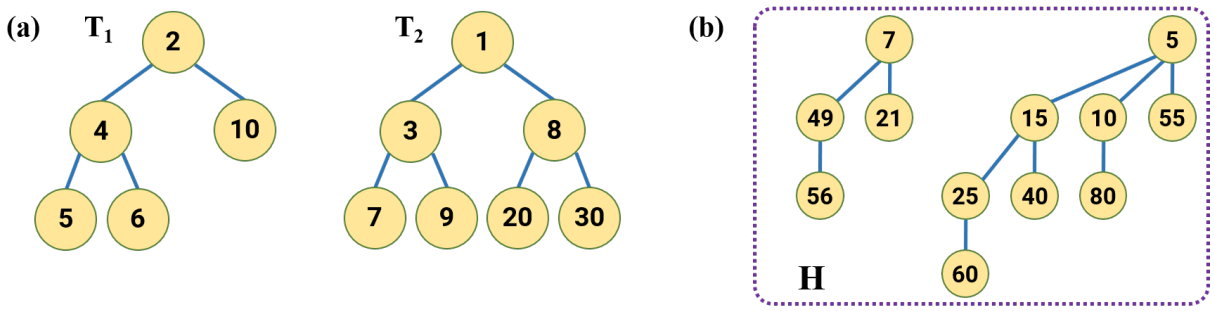
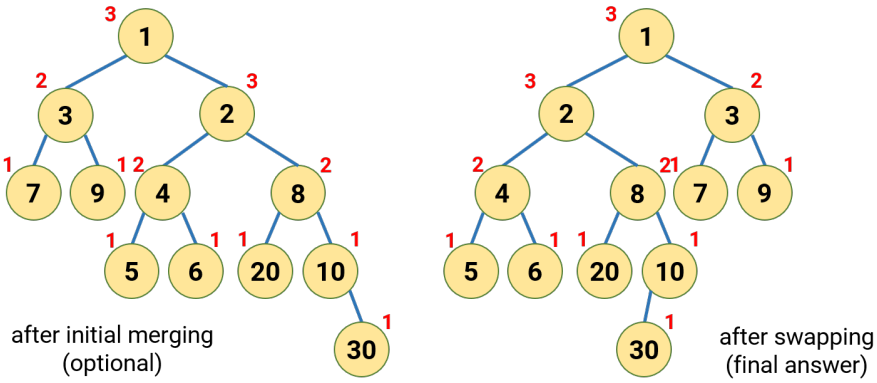
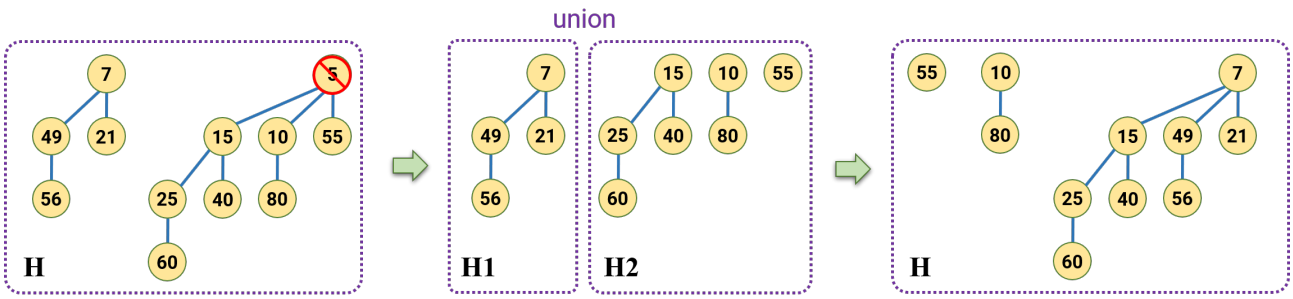


Figure 2: (a) Leftist tree and (b) Binomial heap

7. Show the resulting tree after **merging** the two leftist trees, T_1 and T_2, shown in Figure 2(a). Ensure the leftist property is maintained. (Hint: draw two trees, one after recursive merging, and one after the final swapping, to possibly receive partial credit) (8%)



8. Perform the **Extract-Min** operation on the **Binomial Heap** shown in Figure 2(b). (Hint: show your steps to possibly receive partial credit) (8%)



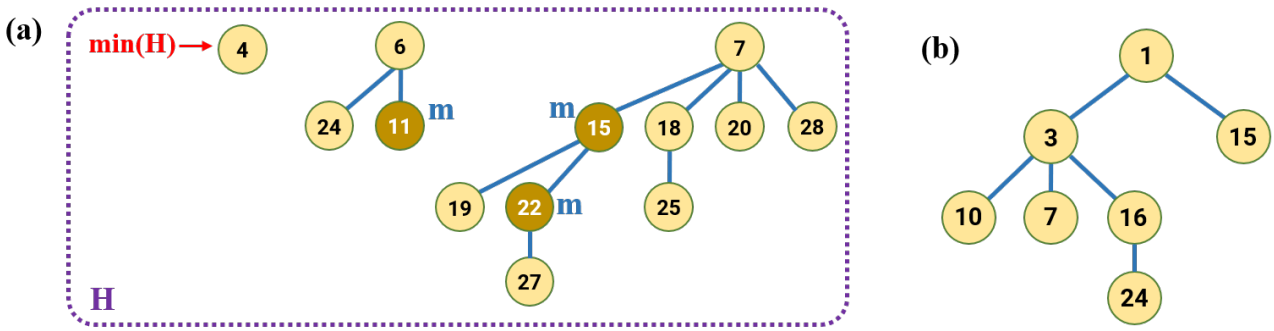
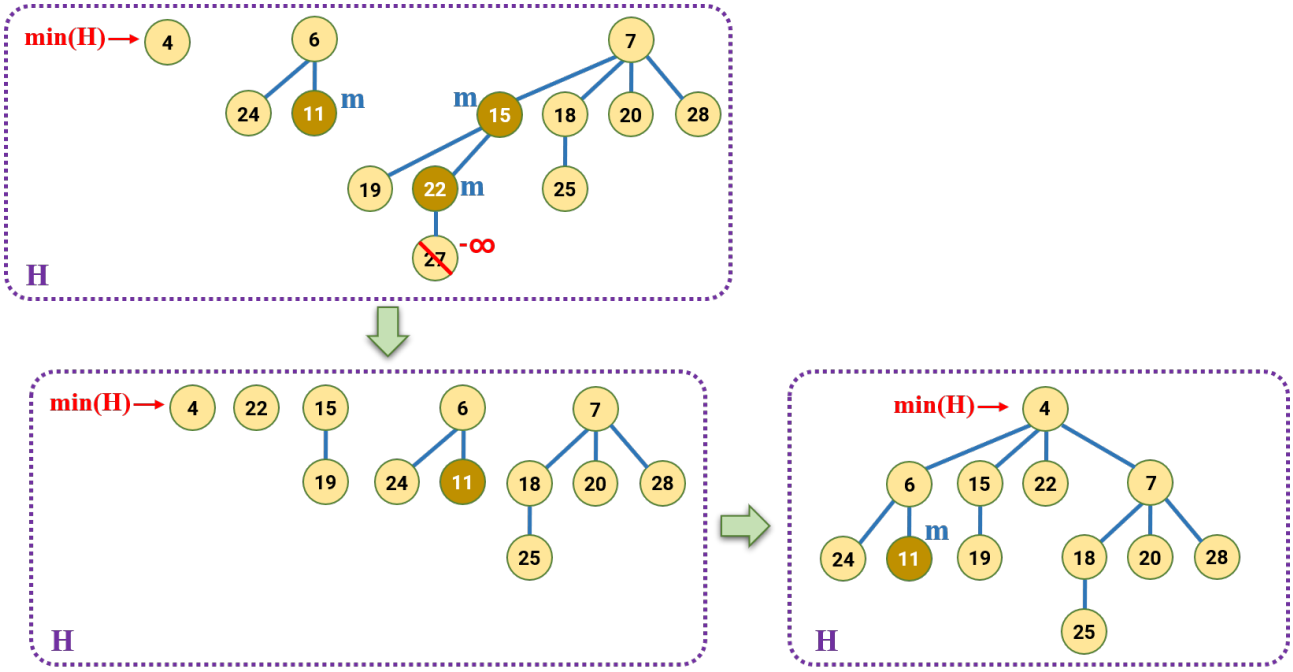
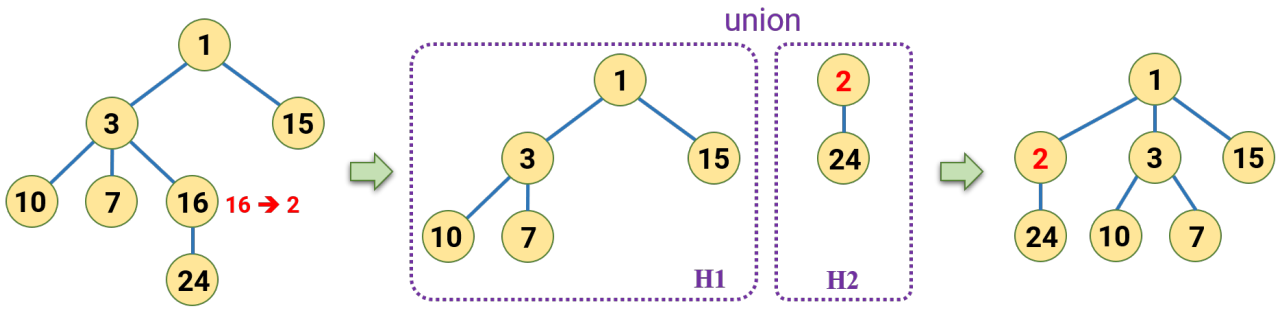


Figure 3: (a) Fibonacci heap and (b) pairing heap

9. Perform the **Delete (27)** operation on the **Fibonacci Heap** shown in Figure 3(a). Your figure should highlight the node markings. (Hint: show your steps to possibly receive partial credit) (8%)



10. Perform the **Decrease-Key (16 → 2)** operation to the **Pairing Heap** shown in Figure 3(b). (Hint: show your steps to possibly receive partial credit) (8%)